

# División del trabajo Etapa 2

---

| Módulo                                     | Estudiante |
|--------------------------------------------|------------|
| Configuración, Precarga y Build System     | Hermes     |
| Protocolo y Serialización                  | Sebas      |
| Motor de Red con pthread                   | Jose       |
| Conmutación y Propagación (Lógica Central) | Marina     |
| Nodos Host y CLI                           | Daniel     |

## Fecha ideal para terminar módulos y comenzar a probar integración

Antes del jueves **24** de septiembre.

## Tareas de cada estudiante

### Tareas en Conjunto

- ☐ Probar todo el sistema de extremo a extremo.
- ☐ **Depuración de concurrencia:** Correr las pruebas usando AddressSanitizer y ThreadSanitizer para asegurar que no existan fugas de memoria ni *race conditions* en la MMU.
- ☐ La MMU usa un `uint32_t`. No se van a usar strings como "eth0". Se va a guardar y buscar directamente el File Descriptor del socket abierto (como si fuera la interfaz).

### Persona 1: Configuración, Precarga y Build System

**Archivos:** configParser.c, Makefile y pruebas.

- ☐ Configuración: Programar la lectura de config.txt para extraer la IP local y los vecinos.
- ☐ Precarga: Hacer los llamados iniciales a `saveRoute()` para cargar las conexiones directas en la memoria virtual antes de levantar la red.
- ☐ Compilación: Crear el Makefile centralizado definiendo las reglas para compilar todos los .c con la bandera -pthread.
- ☐ Testing: Hacer un main de pruebas para probar la MMU de 256 bytes y asegurar que no haya desbordamientos.

---

### Persona 2: Protocolo y Serialización

**Archivos:** protocol.c, protocol.h

- ☐ Decodificación: Programar `decodeFrame()`, asegurando limpiar caracteres residuales o saltos de línea (`\n`, `\r`, `\0`) del buffer del socket para no rechazar tramas válidas.
- ☐ Codificación: Añadir la firma en el .h e implementar `encodeFrame()` para transformar el struct en el string exacto `TIPO|IP|DATOS\n`.

- ☐ Conversión: Utilizar solamente `inet_pton` e `inet_ntop` para manejar las IPs en binario de red (evitar `htonl/ntohl`).
- 

### Persona 3: Motor de Red con pthread

**Archivos:** `router.c`, `router.h`

- ☐ Arranque: Programar `initRouterListen()` configurando el socket (`bind`, `listen`) y lanzando el hilo con `pthread_create`.
- ☐ Bucle Pasivo: Programar la rutina del hilo `listening()` usando `recv()`. No toma decisiones lógicas: su única función es recibir bytes y pasar el buffer crudo y el `sockfd` a la función de enrutamiento (De persona 4).
- ☐ Apagado: Programar `endRouterListen()` para cerrar los descriptores de archivo y matar el hilo limpiamente.

### Persona 4: Conmutación y Propagación (Lógica Central)

**Archivos:** `forwarding.c`, `forwarding.h`

- ☐ Protección de Memoria: Declarar e inicializar un `pthread_mutex_t`. Es obligatorio bloquear y liberar el mutex justo antes y después de cada llamada a `saveRoute` o `findRoute` para no corromper la MMU.
- ☐ Anuncios (Generación): Programar la emisión de un mensaje `ANNOUNCE` propio por los sockets activos apenas el router encienda.
- ☐ Propagación: En la función principal `processPacket(buffer, sockfd)`: si recibe `ANNOUNCE` o `ADVERTISE`, guardarlo con `saveRoute()`. Si la ruta resulta ser nueva, generar un `ADVERTISE` y reenviarlo a los demás routers vecinos.
- ☐ Conmutación: Si el paquete recibido es `DATA`, buscar el destino con `findRoute()` y reenviarlo hacia ese FD usando `send()`.

### Persona 5: Nodos Host y CLI

**Archivos:** `listener.c`, `sender.c`, `main.c`, `listener.h`, `sender.h`

- ☐ Receptor: Programar `keepListening()` configurando el socket cliente pasivo, y `showMessage()` para imprimir datos en terminal.
- ☐ Emisor: Programar `sendMessage()` para que el cliente empaquete y mande datos al router.
- ☐ CLI (Interfaz de comandos): Programar `main.c` con el manejo de argumentos (`argc/argv`) para que el usuario decida si el ejecutable arranca en modo router, emisor o receptor.